

Aggregates are named tuples

“Call him Voldemort, Harry. Always use the proper name for things.”

— J.K. Rowling, *Harry Potter and the Sorcerer's Stone*

O. Changelog

R1:

- Addressed LEWG feedback in "V. Design decisions and LEWG questions"
- Switched to using `std::element_count` rather than specializing `std::tuple_size`
- Prototype with tests for libstdc++: [PR](#)
- [Online Playground at Godbolt](#)

R0:

- Initial version
- "EWG agrees that this is a valid library feature, and LEWG should consider P2141. EWG does not believe this conflicts with any language facilities, current or proposed." SF:9 F:16 N:1 A:0 SA: 0
- "LEWG are interested in P2141R0 but want to see standard library implementation experience due to the additional overload of `std::get`." SF:13 F:8 N:1 A:0 SA: 0

I. Quick Introduction

In C++ we have:

- tuples - types that provide access to members by index. Those are useful for generic programming
- aggregates - types with named fields. Those are just easy to use.

This paper was inspired by multiple years of experience with [PFR/magic_get library](#). The core idea of this paper is to add functionality to some aggregates so that they could behave as tuples.

II. Motivation and Examples

`std::tuple` and `std::pair` are great for generic programming, however they have disadvantages. First of all, code that uses them becomes barely readable. Consider two definitions:

```
struct auth_info_aggregate {
    std::int64_t id;
    std::int64_t session_id;
    std::int64_t source_id;
    std::time_t valid_till;
};

using auth_info_tuple = std::tuple<
    std::int64_t,
    std::int64_t,
    std::int64_t,
    std::time_t
>;
```

Definition via structure is much more clear. Same story with usages: `return std::get<1>(value);` vs. `return value.session_id;`

Another advantage of aggregates is a more efficient copy, move construction and assignments:

```
template <class T>
constexpr bool validate() {
    static_assert(std::is_trivially_move_constructible_v<T>);
    static_assert(std::is_trivially_copy_constructible_v<T>);
    static_assert(std::is_trivially_move_assignable_v<T>);
    static_assert(std::is_trivially_copy_assignable_v<T>);
    return true;
}

constexpr bool tuples_fail = validate<auth_info_tuple>(); // Fails majority of checks
constexpr bool aggregates_are_ok = validate<auth_info_aggregate>();
```

Because of the above issues many coding guidelines **recommend to use aggregates instead of tuples**.

However at the moment aggregates fail when it comes to the functional like programming:

```

namespace impl {
    template <class Stream, class Result, std::size_t... I>
    void fill_fields(Stream& s, Result& res, std::index_sequence<I...>) {
        (s >> ... >> std::get<I>(res));
    }
}

template <class T>
T ExecuteSQL(std::string_view statement) {
    std::stringstream stream;
    // some logic that feeds data into stream

    T result;
    impl::fill_fields(stream, result, std::make_index_sequence<std::tuple_size_v<T>>());
    return result;
}

constexpr std::string_view query = "SELECT id, session_id, source_id, valid_till FROM auth";
const auto tuple_result = ExecuteSQL<auth_info_tuple>(query);
const auto aggregate_result = ExecuteSQL<auth_info_aggregate>(query); // does not compile

// Playground https://godbolt.org/z/y49lya

```

By bringing the functionality of tuples into aggregates we get all the advantages of tuples without losing advantages of aggregates. We get **named tuples**.

III. The Idea

Make `std::get`, `std::tuple_element` and `std::tuple_size/std::element_count` work with aggregates. This also makes `std::tuple_element_t`, `std::apply`, `std::tuple_cat` and `std::make_from_tuple` usable with aggregates.

IV. Interaction with other papers

[P1061 "Structured Bindings can introduce a Pack"](#) makes it really simple to implement the ideas proposed in this paper. For example elements count detection could be implemented as:

```

template <class T>
constexpr std::size_t fields_count() {
    auto [...x] = T();
    return sizeof...(x);
}

```

[P1061](#) is not a requirement for this paper acceptance. Same logic could be implemented as a compiler built-in or even via some metaprogramming tricks, as in [PFR/magic_get_library](#).

There may be concerns, that proposed functionality may hurt [N4818 "C++ Extensions for Reflection"](#) adoption, as some of functionality becomes available without reflection. Experience with [PFR/magic_get_library](#) shows that `std::get` and `std::tuple_size` functions cover only very basic cases of reflection. We still need reflection for trivial things, like serialization to JSON, because only reflection gives us field names of the structure.

Parts of [P1858R1 "Generalized pack declaration and usage"](#) address some of the ideas of this paper on a language level and give simple to use tools to implement ideas of this paper. However this paper brings capabilities symmetry to the standard library, shows another approach to deal with field access by index and allows existing user code to work out-of-the-box with aggregates:

C++20	This paper	P1858
<pre> // Works only with tuples // int foo(auto value) { if (!std::get<10>(value)) { return 0; } return std::apply(function, value); } </pre>	<pre> // Works with tuples and aggregates // No code change required int foo(auto value) { if (!std::get<10>(value)) { return 0; } return std::apply(function, value); } </pre>	<pre> // Works with tuples and aggregates // Users are forced to rewrite code int foo(auto value) { if (!value::[10]) { return 0; } return std::invoke(function, value </pre>
<pre> template <class T> auto portable_function(const T& value) { // Works with tuples since C++11 return std::get<2>(value); } </pre>	<pre> template <class T> auto portable_function(const T& value) { // Works with tuples since C++11 and with aggregates return std::get<2>(value); } </pre>	<pre> template <class T> auto portable_function(const T& value) #ifdef __cpp_generalized_packs // Works with tuples and aggregate return value::[2]; #else // Works with tuples since C++11 return std::get<2>(value); #endif } </pre>

V. Design decisions and LEWG questions.

A. LEWG: Does it affect the user-customized structured bindings?

Good news: no, it does not affect the user-customized structured bindings. The user already specialized the `std::tuple_size` for its type, `std::tuple_size` specialization from this proposal is less specialized. Online playground: <https://godbolt.org/z/Pxnybcv6y>.

Bad news: R0 of the proposal does affect all the non-customized structured bindings <https://godbolt.org/z/dro9nGEd7>. It is because R0 of the proposal specializes the `std::tuple_size` and the [dcl.struct.bind] p4 uses `std::tuple_size` to distinguish between "customized" and "compiler implemented" [dcl.struct.bind] p5 structured bindings.

The solution is to either adjust the [dcl.struct.bind] p4 to not take into account the added specialization of `std::tuple_size` or to not specialize the `std::tuple_size` and use a separate function for getting the elements count from an aggregate.

See the "X. To specialize or not to specialize `std::tuple_size`" below for more discussion.

B. LEWG: does it change the meaning of code with usages of `tuple_cat` on string literals?

`std::tuple_cat` could be used with parameters that satisfy the *tuple-like* concept. According to [tuple.like] the concept is not satisfied for string literals in C++23. So the meaning does not change because string literals can not be used right now with `std::tuple_cat`. Godbolt playground: <https://godbolt.org/z/cxc8s5qeG>.

With this proposal code like `std::tuple_cat(aggregate1{"foo"}, aggregate2{"bar", "baz"})` would produce `std::tuple<const char*, const char*, const char*>{"foo", "bar", "baz"}` if aggregate1 and aggregate2 have `const char*` elements, or would fail to compile because `std::tuple<char[4], char[4], char[4]>` could not be constructed. Godbolt playground: <https://godbolt.org/z/fxM8za3WG>.

C. LEWG: Null termination of string literals can lead to surprising `tuple_cat` behavior

`std::tuple_cat("a", "b")` would produce a `std::tuple<char, char, char, char>{'a', '\0', 'b', '\0'}`. That might be surprising for first time, but that's what the code asked for: treat two parameters as tuples and concatenate those. If that behavior is undesired, then it could be disabled by prohibiting arrays in *tuple-like*.

D. LEWG: Does it change the behavior of user `tuple_flatten`-like functions?

Consider the following snippet:

```
int main() {
    std::tuple<
        std::tuple<std::tuple<Noisy, Noisy>>,
        std::tuple<std::tuple<Noisy>>
    > t;

    auto x = tuple_flatten(t);
    static_assert(std::is_same_v<decltype(x), std::tuple<Noisy, Noisy, Noisy>>);
    static_assert(!std::is_same_v<decltype(x), std::tuple<int, short, int, short, int, short>>);
}
```

Implementation of the `tuple_flatten` could use one of the following mechanics to decide when to stop flattening:

1. Internals of the `tuple_flatten` accept only `std::tuple`, so the `tuple_flatten` works only with `std::tuple`.
2. Internals of the `tuple_flatten` use *tuple-like* concept.
3. Internals of the `tuple_flatten` SFINAE on `std::tuple_size<T>::value`

tuple-like concept is exposition only, users should not use it.

So the behavior changes if we specialize the `std::tuple_size` and users SFINAE on `std::tuple_size` or `std::tuple_size_v`. In many cases such behavior change would be detected at compile time, however may be some cases when the compilation would succeed and the code silently changes behavior.

See the "X. To specialize or not to specialize `std::tuple_size`" below for more discussion.

E. What about customization?

The intent of this proposal it to follow the structured bindings behavior and customizations. So if the user customized that the aggregate with three elements of type `int` has 2 elements of type `short` - the `std::get` should follow.

The alternative is to skip all the customizations and just do the aggregate reflection as is. This contradicts the Reflections, as the main steam is the "value based reflection" rather than template based.

F. Why not just use structured bindings?

1. There is no way to SFINAE on structured binding which is important for implementing generic functions. Some kind of `std::element_count`, or `std::tuple_size` specializations or public concept is required.
2. There is no simple and fast way to get elements count. Some kind of `std::element_count` function is required or [P1061 "Structured Bindings can introduce a Pack"](#).
3. No ready to use functions to work with aggregates as with tuples, like `std::tuple_cat`. *tuple-like* should be adjusted or new functions should be provided.
4. Existing user and standard library implementations of algorithms rely on `std::get`. To ease the migration from `std::tuple` to aggregates those functions should be also provided.

In other words: for smooth use of structured binding in a generic programming we need something close to the changes proposed in this paper.

G. Impact on the Standard Library

1. `std::tuple` becomes constructible and assignable from aggregates of the matching elements count.
2. `std::pair` becomes constructible and assignable from aggregates of 2 elements.
3. `std::tuple_cat` can concatenate elements of aggregates into `std::tuple`.
4. `std::apply` can apply elements of an aggregate to a function.

5. `std::make_from_tuple<T>` can construct type `T` from aggregate.
6. `std::tuple` becomes comparable with aggregates of matching elements count.

Functions that explicitly require instance of `std::tuple` or `std::pair` are not affected (`basic_common_reference`, `common_type`, `format_kind`, 'm' *range-type* specifier).

X. To specialize or not to specialize `std::tuple_size`

	Specialize <code>std::tuple_size</code>	Provide a separate <code>std::element_count</code>
Does not affect the user-customized structured bindings	✓	✓
Does not change the meaning of code with <code>std::tuple_cat</code>	✓	✓
Does not change the behavior of user <code>tuple_flatten</code> -like functions	✗	✓
User tuple-code works with aggregates out of the box	✓ if SFINAEs on <code>std::tuple_size</code>	✗ (requires explicit <code>std::element_count</code> usage)
Gives a way to explicitly allow reflection of aggregates	✓ (SFINAE on <code>std::is_aggregate_v</code>)	✓ (SFINAE on <code>std::tuple_size</code>)
Is it customizable?	✓	✓

Assuming that the `tuple_flatten`-like functions are common and could lead to silent behavior change this revision concentrates on `std::element_count` approach. If LEWG decides that the paper should proceed with `std::tuple_size` specialization, the structured binding wording [dcl.struct.bind] p4 should be changed to something like the following:

Otherwise, if the qualified-id `std::tuple_size<E>` names a complete class type **not inherited from `std::element_count<E>` and** with a member named `value`, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression and the number of elements in the identifier-list shall be equal to the value of that expression.

VII. Wording

After adjusting `yyymm` (below) so as to denote this proposal's month of adoption, insert the following line among the similar directives following [version.syn]/2:

```
#define __cpp_lib_aggregate_as_tuple yyymmL // also in <utility>, <tuple>
```

Add to the bottom of [utility.syn], right before the last closing bracket:

```
// [utility.aggregate], tuple-like access to aggregate
template<tuple-like T>
    using element_count = see below;

template<tuple-like T>
    constexpr size_t element_count_v = element_count<T>::value;

template<size_t I, tuple-like T> struct tuple_element;

template<size_t I, tuple-like T>
    constexpr tuple_element_t<I, T>& get(T&) noexcept;
template<size_t I, tuple-like T>
    constexpr tuple_element_t<I, T>&& get(T&&) noexcept;
template<size_t I, tuple-like T>
    constexpr tuple_element_t<I, const T>& get(const T&) noexcept;
template<size_t I, tuple-like T>
    constexpr tuple_element_t<I, const T>&& get(const T&&) noexcept;

template<class T, tuple-like TupleLike>
    constexpr T& get(TupleLike&) noexcept;
template<class T, tuple-like TupleLike>
    constexpr T&& get(TupleLike&&) noexcept;
template<class T, tuple-like TupleLike>
    constexpr const T& get(const TupleLike&) noexcept;
template<class T, tuple-like TupleLike>
    constexpr const T&& get(const TupleLike&&) noexcept;
```

Add after [pair.piecewise]:

22.3.6 Tuple-like access to aggregate [utility.aggregate]

```
template<tuple-like T>
    using element_count = see below;
    The element_count meets the Cpp17UnaryTypeTrait requirements ([meta.rqmts]) with a base characteristic of integral_constant<size_t, N> for N being tuple_size<T>::value if tuple_size<remove_cv_ref_t<T>> names a complete class type with a member named value; otherwise N is the number of non static data members in remove_cv_ref_t<T>.

template<size_t I, tuple-like T> struct tuple_element;
    Let TE denote the type of the Ith aggregate element of T, where indexing is zero-based. Specialization meets the Cpp17TransformationTrait requirements ([meta.rqmts]) with a member typedef type that names the type TE.
```

```

template<size_t I, tuple-like T>
constexpr tuple_element_t<I, T>& get(T& t) noexcept;
template<size_t I, tuple-like T>
constexpr tuple_element_t<I, T>&& get(T&& t) noexcept;
template<size_t I, tuple-like T>
constexpr tuple_element_t<I, const T>& get(const T& t) noexcept;
template<size_t I, tuple-like T>
constexpr tuple_element_t<I, const T>&& get(const T&& t) noexcept;
    Let  $v_0, \dots, v_{n-1}$  be the identifiers introduced by structured binding declaration
    auto  $[v_0, \dots, v_{n-1}] = \text{std::forward}<\text{decltype}(t)>(t);$ ,
    where  $n$  is equal to element_count<T>::value.
    Mandates:  $I < \text{element\_count}<T>::\text{value}$ .
    Returns: A reference to the  $v_I$ .

template<class T, tuple-like TupleLike>
constexpr T& get(TupleLike& t) noexcept;
template<class T, tuple-like TupleLike>
constexpr T&& get(TupleLike&& t) noexcept;
template<class T, tuple-like TupleLike>
constexpr const T& get(const TupleLike& t) noexcept;
template<class T, tuple-like TupleLike>
constexpr const T&& get(const TupleLike&& t) noexcept;
    Let  $v_0, \dots, v_{n-1}$  be the identifiers introduced by structured binding declaration
    auto  $[v_0, \dots, v_{n-1}] = \text{std::forward}<\text{decltype}(t)>(t);$ ,
    where  $n$  is equal to element_count<T>::value.
    Mandates: Exactly one of the  $v_0, \dots, v_{n-1}$  identifiers has type T.
    Returns: A reference to the identifier corresponding to the type T.

```

Adjust [tuple.like]:

```

template<class T>
concept tuple-like = see below;           // exposition only
    A type T models and satisfies the exposition-only concept tuple-like
    if remove_cvref_t<T> is
    a specialization of array, pair, tuple, or ranges::subrange
    the structured binding declaration auto  $[v_0, \dots, v_{N-1}] = \text{declval}<T>();$  would be well formed for some N
    and none of the  $v_0, \dots, v_{N-1}$  refers to a bitfield.

```

Change the `tuple_size_v` usages to `element_count_v`:

[tuple.syn]:

```

template<class T>
concept pair-like =                       // exposition only
    tuple-like<T> && element_count_v<T> tuple_size_v<remove_cvref_t<T>> == 2;

```

[tuple.cnstr]:

```

template<tuple-like UTuple>
constexpr explicit(see below) tuple(UTuple&& u);

    Let I be the pack 0, 1, ..., (sizeof...(Types) - 1).
    Constraints:
    - different-from<UTuple, tuple> ([range.utility.helpers]) is true,
    - remove_cvref_t<UTuple> is not a specialization of ranges::subrange,
    - sizeof...(Types) equals element_count_v<UTuple> tuple_size_v<remove_cvref_t<UTuple>>,

```

[tuple.assign]:

```

template<tuple-like UTuple>
constexpr tuple& operator=(UTuple&& u);

    Constraints:
    - different-from<UTuple, tuple> ([range.utility.helpers]) is true,
    - remove_cvref_t<UTuple> is not a specialization of ranges::subrange,
    - sizeof...(Types) equals element_count_v<UTuple> tuple_size_v<remove_cvref_t<UTuple>>, and,
    - is_assignable_v<Ti&, decltype(get<i>(std::forward<UTuple>(u)))> is true for all i.

    Effects: For all i, assigns get<i>(std::forward<UTuple>(u)) to get<i>(*this).
    Returns: *this.

```

```

template<tuple-like UTuple>
constexpr const tuple& operator=(UTuple&& u) const;

    Constraints:
    - different-from<UTuple, tuple> ([range.utility.helpers]) is true,
    - remove_cvref_t<UTuple> is not a specialization of ranges::subrange,
    - sizeof...(Types) equals element_count_v<UTuple> tuple_size_v<remove_cvref_t<UTuple>>, and,

```

[tuple.creation]:

```

template<tuple-like... Tuples>
constexpr tuple<CTypes...> tuple_cat(Tuples&&... tpls);

    Let n be sizeof...(Tuples).

```

For every integer $0 \leq i < n$:

- Let T_i be the i th type in Tuples.
- Let U_i be `remove_cvref_t<T_i>`.
- Let t_{pi} be the i th element in the function parameter pack `tpls`.
- Let S_i be `tuple_size_v<element_count_v<U_i>`.

[tuple.apply]:

```
template<class F, tuple-like Tuple>
constexpr decltype(auto) apply(F&& f, Tuple&& t) noexcept(see below);
Effects: Given the exposition-only function:
namespace std {
    template<class F, tuple-like Tuple, size_t... I>
    constexpr decltype(auto) apply_impl(F&& f, Tuple&& t, index_sequence<I...>) {
        // exposition only
        return INVOKE(std::forward<F>(f), get<I>(std::forward<Tuple>(t))...); // see [func.require]
    }
}
```

Equivalent to:

```
return apply-impl(std::forward<F>(f), std::forward<Tuple>(t),
    make_index_sequence<element_count_v<Tuple>tuple_size_v<remove-reference-t<Tuple>>{}>{}>);
```

Remarks: Let I be the pack $0, 1, \dots, (\text{element_count_v}<\text{Tuple}>\text{tuple_size_v}<\text{remove_reference_t}<\text{Tuple}>> - 1)$.

The exception specification is equivalent to:

```
noexcept(invoker(std::forward<F>(f), get<I>(std::forward<Tuple>(t))...))
```

```
template<class T, tuple-like Tuple>
constexpr T make_from_tuple(Tuple&& t);
Mandates: If element_count_v<Tuple>tuple_size_v<remove-reference-t<Tuple>> is 1, then
reference_constructs_from_temporary_v<T, decltype(get<0>(declval<Tuple>()))> is false.
```

Effects: Given the exposition-only function:

```
namespace std {
    template<class T, tuple-like Tuple, size_t... I>
    requires is_constructible_v<T, decltype(get<I>(declval<Tuple>()))...>
    constexpr T make_from_tuple_impl(Tuple&& t, index_sequence<I...>) { // exposition only
        return T(get<I>(std::forward<Tuple>(t))...);
    }
}
```

Equivalent to:

```
return make-from-tuple-impl<T>(
    std::forward<Tuple>(t),
    make_index_sequence<element_count_v<Tuple>tuple_size_v<remove-reference-t<Tuple>>{}>{}>);
```

[tuple.rel]:

```
template<class... TTypes, class... UTypes>
constexpr bool operator==(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
template<class... TTypes, tuple-like UTuple>
constexpr bool operator==(const tuple<TTypes...>& t, const UTuple& u);
```

For the first overload let `UTuple` be `tuple<UTypes...>`.

Mandates: For all i , where $0 \leq i < \text{sizeof}...(TTypes)$, `get<math>i</math>(t) == get<math>i</math>(u)` is a valid expression.
`sizeof...(TTypes)` equals `tuple_size_v<element_count_v<UTuple>`.

Preconditions: For all i , `decltype(get<math>i</math>(t) == get<math>i</math>(u))` models `boolean-testable`.

Returns: true if `get<math>i</math>(t) == get<math>i</math>(u)` for all i , otherwise false.

[Note 1: If `sizeof...(TTypes)` equals zero, returns true.
 - end note]

Remarks:

- The elementary comparisons are performed in order from the zeroth index upwards.
 No comparisons or element accesses are performed after the first equality comparison that evaluates to false.
- The second overload is to be found via argument-dependent lookup ([basic.lookup.argdep]) only.

```
template<class... TTypes, class... UTypes>
constexpr common_comparison_category_t<synth-three-way-result<TTypes, UTypes>...>
operator<=>(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
template<class... TTypes, tuple-like UTuple>
constexpr common_comparison_category_t<synth-three-way-result<TTypes, Elms>...>
operator<=>(const tuple<TTypes...>& t, const UTuple& u);
```

For the second overload, `Elms` denotes the pack of types `tuple_element_t<0, UTuple>`, `tuple_element_t<1, UTuple>`, ..., `tuple_element_t<tuple_size_v<element_count_v<UTuple> - 1, UTuple>`.

Effects: Performs a lexicographical comparison between `t` and `u`.

If `sizeof...(TTypes)` equals zero, returns `strong_ordering::equal`.

Otherwise, equivalent to:

```
if (auto c = synth-three-way(get<0>(t), get<0>(u)); c != 0) return c;
return ttail <=> utail;
```

where `rtail` for some `r` is a tuple containing all but the first element of `r`.

Remarks: The second overload is to be found via argument-dependent lookup ([basic.lookup.argdep]) only.

VI. Acknowledgments

Many thanks to Barry Revzin for writing P1858 and providing early notes on this paper.